Data Sentinel AI: An AI-Powered Multi-Channel Data Loss Prevention System

Muhammed Dulfazal *26MAG0081*
Rohen Paul J *26MAG0087*
Adithya Ram *26MAG0089*

Guide: Dr. Gayathri A.
Course: Python Programming (PAAML503)

***Abstract*—Sensitive organisational data such as customer records, salary sheets, payment-card numbers, identity numbers and access keys can leave an endpoint through many routes: USB drives, cloud sync folders, network shares, website uploads and pastes (including generative-AI chat tools), e-mail, and over-shared cloud files. Rule-only data loss prevention (DLP) tools are explainable but usually cover a single channel and miss unusual behaviour. This paper presents Data Sentinel AI, a Python-based endpoint DLP system that passes every transfer on every channel through one pipeline: metadata extraction, rule-based content classification validated with the Luhn checksum, context analysis, hybrid anomaly detection that combines an Isolation Forest with a frequency heuristic, a weighted 0–100 risk score, and a policy engine that returns ALLOW, WARN or BLOCK together with plain-English reasons. Every block raises an alert, and every decision is appended to a SHA-256 hash-chained, tamper-evident audit log that is shown on a Streamlit dashboard. Six channels are implemented: removable drives, cloud sync folders and network shares through folder monitoring with quarantine; websites through a Manifest V3 browser extension and a local scan API; desktop e-mail through an SMTP gateway; and existing Google Drive sharing through a Drive API scanner. On 5,000 synthetic transfer events the Isolation Forest reached an F1-score of 94.4% and a ROC-AUC of 99.96%, against 66.7% and 79.4% for the frequency heuristic, and the hybrid raised recall to 97.3%. The Luhn check reduced false card-number alarms on 10,000 random numbers from 10,000 to 980. All ten end-to-end scenarios produced the expected decisions and all 125 automated tests passed.**

***Index Terms*—Data loss prevention, insider threat detection, anomaly detection, isolation forest, sensitive data classification, explainable risk scoring, tamper-evident audit log, browser extension.**

I. Introduction

Organisations hold large volumes of sensitive information: customer records, salary sheets, payment-card numbers, national identity numbers and access keys. Data leakage occurs when such information leaves the organisation without authorisation, either by mistake or on purpose. Insiders are a major source of such incidents because they already have legitimate access; recent reviews describe insider threat detection as a paramount concern and report that current detection strategies are acknowledged to be ineffective [1], [2]. At the same time the number of exit routes keeps growing. Besides USB drives and e-mail, employees now upload files to personal cloud storage, share links publicly, and paste text into generative-AI chat tools, whose privacy and security concerns are themselves the subject of recent surveys [3]. Protecting data in cloud environments also remains a primary concern [4].

Traditional data loss prevention (DLP) relies on fixed rules such as keyword lists or a ban on all USB devices. Fixed rules are easy to explain but they miss unusual behaviour, for example an employee copying many ordinary files late at night, and most deployments cover only one channel. Machine-learning detectors capture behaviour but are often opaque, and the security literature stresses that decisions which cannot be explained are hard for analysts to trust [5]–[7]. A practical DLP system therefore needs both: the precision and transparency of rules for content, and the adaptability of anomaly detection for behaviour, applied consistently wherever data can leave.

This paper presents **Data Sentinel AI**, an AI-assisted DLP system for a single endpoint. Every transfer on every channel is treated as an event and processed by one pipeline: metadata extraction, content classification, context analysis, anomaly detection, risk scoring, policy enforcement, response and audit. The main contributions are:

- a single, thread-safe pipeline shared by six channels: USB drives, cloud sync folders, network shares, websites (uploads, drag-and-drop and pastes), desktop e-mail, and files already shared in Google Drive;
- a hybrid anomaly detector that takes the stronger of an unsupervised Isolation Forest and a transparent frequency heuristic, so the system works before any history exists;
- an explainable weighted risk score (0–100) and a policy engine with hard rules, where every decision carries plain-English reasons and stores only the type and count of matched data, never the values;
- a SHA-256 hash-chained audit log that detects edited or deleted records, and a live monitoring dashboard;
- an evaluation with a synthetic exfiltration benchmark, a comparison of five detectors, a Luhn false-alarm test, ten end-to-end scenarios and 125 automated tests.

Section II surveys recent IEEE journal literature. Section III describes the architecture. Section IV presents the proposed work: working procedure, inputs, algorithms, pseudocode, and the results with their explanations. Section V concludes and outlines future work.

II. Literature Survey

The survey covers IEEE journal articles published between 2022 and 2026, with an emphasis on survey and review papers. They are grouped into six themes that correspond to the stages of the proposed pipeline. Table I summarises the works and their relevance to this project.

A. Insider Threat Detection

Alzaabi and Mehmood [1] review insider threat detection with a focus on classical machine-learning approaches and their limitations, and present deep learning and natural-language-processing methods as promising alternatives. They

recommend time-series techniques, because insider behaviour has temporal patterns, and large-language-model approaches for textual context. Whitelaw *et al.* [2] review 54 peer-reviewed articles from a practitioner perspective in U.K. financial services and identify five themes: behavioural indicators, information security behaviours, technical controls, insider threat strategies and regulation. They note that current detection strategies are acknowledged as ineffective, as shown by high-profile insider incidents and cyber data-loss cases. Saeed Alketbi and Mehmood [7] survey explainable AI (XAI) for insider threat detection, examining tools such as SHAP and LIME that reveal feature contributions. They report persistent challenges: class imbalance, heterogeneous data streams and the absence of standard explainability metrics.

B. Anomaly Detection and Isolation Forest

Umar and Jordanov [8] follow the PRISMA methodology to survey machine-learning anomaly detection, covering methods from isolation forests and one-class SVMs to autoencoders and graph neural networks. Their comparison finds that supervised and deep models reach high accuracy when ample labelled data exists, whereas unsupervised and hybrid methods are more robust against novel attacks and changing behaviour. Al-Shehari *et al.* [9] apply the Isolation Forest to the heavily imbalanced CERT insider-threat dataset, addressing imbalance at the algorithm level instead of by resampling. They evaluate several contamination ratios and report an accuracy of 98%. Peccatiello *et al.* [10] build a stream-based framework with periodic retraining and compare Isolation Forest, Elliptic Envelope and Local Outlier Factor; the Isolation Forest gave the best results, with a recall of 0.78 for malicious and 0.80 for benign events. Xu *et al.* [11] describe the Isolation Forest as arguably the most popular anomaly detector, owing to its effectiveness and scalability, but show that its linear, axis-parallel isolation misses hard anomalies. They propose a deep isolation forest based on random neural representations.

C. Sensitive Data Identification

Gambarelli *et al.* [12] release SPEDAC, an annotated English resource for sensitive personal data with three tasks of increasing difficulty: binary detection, five macro-categories and 61 fine-grained categories. Transformer classifiers perform well on the first two but struggle with the fine-grained categories. Oreščanin *et al.* [13] discuss how the GDPR changed the handling of personal identifiable information (PII), and propose a metadata model that supports the discovery, matching and removal of PII in data lakes.

D. Explainability and Trust

Capuano *et al.* [5] analyse more than 300 papers on XAI in cybersecurity, including intrusion detection, malware, phishing, fraud and digital forensics. They observe that explainability improves security practice but can also leave a system more exposed to adversaries. Rjoub *et al.* [6] review XAI for network and system security using a systematic classification of threats, and discuss the limitations of current methods. Syed *et al.* [14] survey the zero-trust security paradigm and the techniques for realising it: authentication, access control, trust and risk computation, micro-segmentation and security automation.

E. New Exit Channels and Exfiltration

Golda *et al.* [3] survey the privacy and security challenges of generative AI and the strategies needed from users, developers, institutions and policymakers. Oz *et al.* [15] give an overview of the security and privacy risks of modern browser application programming interfaces; this is relevant here because a browser-side DLP control is built on these interfaces. Gupta *et al.* [4] systematically review secure data storage and sharing techniques for cloud environments, where data protection is the primary concern. Cai *et al.* [16] detect advanced-persistent-threat exfiltration during network transfers using packet and byte transfer-rate metrics and an ensemble deep-learning tree model, arguing that continuous monitoring is needed once preventive measures fail.

TABLE I
SUMMARY OF THE SURVEYED IEEE JOURNAL LITERATURE (2022–2026)

Ref.	Year	Type	Focus	Relevance to this work
[1]	2024	Review	ML, deep learning and NLP for insider threat detection	Need for data-driven detection beyond fixed rules
[2]	2024	Review	54 articles: behavioural indicators, controls, regulation	Current insider-threat strategies reported ineffective
[7]	2025	Survey	XAI (SHAP, LIME) for insider threat detection	Interpretability builds analyst trust
[8]	2026	Survey	Survey of ML anomaly detection (PRISMA)	Unsupervised and hybrid methods robust to novel attacks
[9]	2023	Article	Isolation Forest on the CERT insider-threat dataset	Handles class imbalance at algorithm level
[10]	2023	Article	IF, Elliptic Envelope, LOF on data streams	Isolation Forest gave the best recall
[11]	2023	Article	Deep Isolation Forest with random representations	Limits of axis-parallel isolation; future upgrade path
[12]	2023	Article	SPEDAC corpus for sensitive personal data	Fine-grained sensitive-data classes remain hard
[13]	2024	Article	Metadata model for PII in data lakes (GDPR)	PII must be discovered and tracked by metadata
[5]	2022	Survey	XAI across 300+ cybersecurity papers	Explainability is needed but can expose models
[6]	2023	Survey	XAI for network and system security	Open challenges of XAI in security
[14]	2022	Survey	Zero-trust tenets, trust and risk computation	Trust and risk computation techniques
[3]	2024	Survey	Privacy and security of generative AI	Privacy and security challenges of generative AI
[15]	2025	Article	Security and privacy of modern browser APIs	Browser APIs carry intrinsic security and privacy risks
[4]	2022	Review	Secure storage and sharing of cloud data	Cloud data protection remains a primary concern
[16]	2025	Article	APT exfiltration detection from network traffic	Continuous monitoring catches exfiltration missed early
[17]	2024	Survey	Forensics and compliance auditing frameworks	Audit trails support investigation and compliance
[18]	2026	Article	Blockchain-based verifiable log auditing (BLAS)	Verifiable, tamper-evident logs; future anchoring

F. Audit and Forensics

Henriques *et al.* [17] survey forensics and compliance auditing for critical infrastructure and propose a reference architecture that unifies the two, since both depend on the same data sources. Huang *et al.* [18] present BLAS, a blockchain-enabled verifiable log audit system whose Merkle-tree-based index supports efficient, fine-grained auditing of log entries.

G. Research Gap

The surveyed works address insider behaviour, anomaly models, sensitive-data recognition, explainability, new exit channels and audit integrity largely as separate problems. The insider-threat models [9], [10] focus on detecting anomalous behaviour in activity data rather than on enforcing a decision at the moment a file or paste leaves the endpoint. The sensitive-data work [12], [13] identifies what is sensitive but not where it is going or who is moving it. Explainability is repeatedly named as a requirement [5]–[7]. Data Sentinel AI addresses this gap by combining content classification, context, an unsupervised anomaly detector and an explainable score in one decision. That decision is enforced on six endpoint channels, including pastes into AI chat tools, and recorded in a tamper-evident log.

III. Architecture Diagram

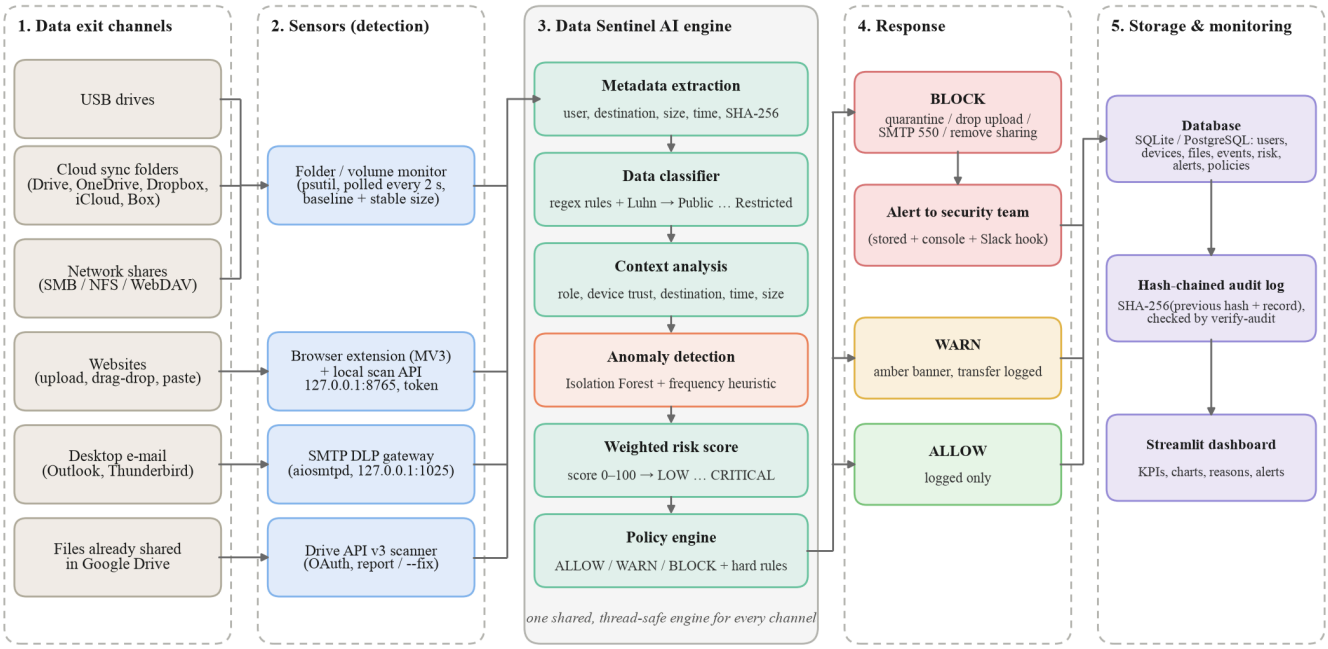


Fig. 1. System architecture of Data Sentinel AI: five layers from the data exit channels to storage and monitoring. All sensors feed one shared engine.

Fig. 1 shows the five layers of the system. **(1) Data exit channels** are the routes data can take: USB drives, cloud sync folders (Google Drive for desktop, OneDrive, Dropbox, iCloud Drive and Box), network shares, websites, desktop e-mail, and files already shared in Google Drive.

(2) Sensors detect a transfer when it happens. A folder and volume monitor polls the operating system every two seconds through psutil. On macOS it confirms removable volumes with diskutil; on Windows it uses the removable flag and the volume label and serial number; on Linux it reads /sys/block and the USB bus. Files already present form a baseline, and a new file is scanned once its size stops changing, while temporary files and operating-system folders are ignored. A Manifest V3 browser extension holds file-input uploads (including the detached inputs used by Google Drive), drag-and-drop and pastes of 20 or more characters, and asks a local scan API on 127.0.0.1:8765. The API requires a random token, sends no CORS headers so ordinary web pages cannot call it, and rejects oversized requests with HTTP 413. An SMTP gateway on 127.0.0.1:1025 receives mail from desktop clients. A Drive API v3 scanner lists files shared with anyone or with outside people or domains.

(3) The Data Sentinel AI engine is shared by all sensors and is thread-safe: each event is one database transaction, so watcher threads, the API and the gateway can run at once without breaking the audit chain. Its stages are metadata extraction, classification, context analysis, anomaly detection, risk scoring and policy evaluation (Section IV). **(4) The response layer** acts on the original channel. A blocked file copy is moved to a quarantine folder with a note that holds the reasons but no content. A blocked upload or paste never reaches the website and a red banner appears. A blocked e-mail is refused with SMTP code 550, and a blocked Drive file loses only its public or external permission. Every block raises an alert, and a dry-run mode decides, logs and alerts without changing anything. **(5) Storage and monitoring** keeps users, devices, files, transfer events, risk assessments, alerts, policies and the audit log in SQLite or PostgreSQL. A Streamlit dashboard presents them (Fig. 8).

IV. Proposed Work

A. Working Procedure

Every transfer follows the same eight steps:

1. **Detect:** a sensor reports a file, attachment, upload, paste or shared file.
2. **Metadata:** user, destination, channel, file name, size, time and SHA-256 hash are recorded. Content is processed in

memory and never stored.

3. **Classify:** file-name and content rules assign a sensitivity level from 0 (Public) to 3 (Restricted).
4. **Context:** user role and history, destination trust, destination type, time and file size are scored from 0 to 1.
5. **Anomaly:** the hybrid detector scores how unusual the transfer is.
6. **Risk:** the six signals are combined into a 0–100 score and a risk level.
7. **Policy and response:** the policy table and hard rules give ALLOW, WARN or BLOCK, which the channel carries out; a BLOCK raises an alert.
8. **Audit:** the event, assessment and reasons are stored and appended to the hash-chained audit log.

B. Input

Each transfer event is made of the data being moved and its context, as listed in Table II.

TABLE II
INPUTS OF A TRANSFER EVENT

Input	Description	Example
Content	File, attachment, pasted text or exported Drive file	customer_payroll_confidential.csv
Channel	Sensor that saw the transfer	usb, browser, email, cloud_sync, drive
Destination	Device, website host or recipient domain	Unknown-USB-77, chatgpt.com, mail:gmail.com
Trust	Registered trust of the destination	trusted / unknown / untrusted
User and role	Who moves the data	carol (contractor)
Timestamp	Time of transfer (evaluated in IST)	Mon 05 Oct 2026, 11:30
History	User's transfers (1 h) and blocks (30 days)	11 transfers in the last hour
Configuration	Company domains, policy table, weights	LOW → ALLOW, HIGH → BLOCK

C. Algorithms

1) Content Classification with the Luhn Checksum

Regular expressions detect sensitive content. Level 3 (Restricted) covers card numbers of 13–19 digits that pass the Luhn check, Aadhaar-style numbers in the form dddd dddd dddd (with look-arounds so the digits of a card number are not counted twice), PAN numbers, U.S. social security numbers, private-key headers, AWS access keys, assignments of passwords, secrets or API keys with values of six or more characters, and the words *restricted*, *top secret* and *strictly confidential*. Level 2 (Confidential) covers *confidential*, payroll and salary terms, customer lists, balance sheets and ten or more e-mail addresses. Level 1 (Internal) is triggered by *internal use only*. File-name rules add credential files (.pem, .key, .pfx, .p12, .env, id_rsa) at level 3, and sensitive keywords and database or backup extensions at level 2. Text formats, Word and PDF files are read; other types are judged by name. Files above 20 MB arriving from the browser are judged by name and size without writing a temporary file. The highest matching level wins, and only the type and count of each match (for example, *credit card number x1*) are kept.

2) Context Signals

With role risk r (admin 0.1, employee 0.4, intern 0.6, contractor 0.7) and b, the number of the user's blocked transfers in the last 30 days, the user signal is

$$U = min(1,\ r + 0.1 \cdot min(b,\ 3)) \quad (1)$$

Trust T is 0 for a trusted destination, 0.7 for unknown and 1.0 for untrusted. The destination signal D ranges from 0 (internal) through 0.6 (external e-mail), 0.7 (USB, web) and 0.8 (cloud, sync folder, messaging, AI chat) to 0.9 (public cloud share) and 1.0 (paste site). The context signal C adds 0.6 on weekends or outside 07:00–20:00 IST and 0.3 for files above 50 MB, capped at 1.

3) Hybrid Anomaly Detection

The feature vector is [hour, sensitivity, size in MB, destination risk, n], where n is the number of transfers by the same user in the last hour. An Isolation Forest with 200 trees and contamination 0.05 is trained without labels. Its raw score s(x) is mapped to a unit score, and combined with the heuristic:

$$u = clip((-s(x) - 0.5) / 0.2,\ 0,\ 1),\ h = min(1,\ n / 10) \quad (2)$$

$$A = max(u,\ h) \quad (3)$$

The model is loaded from disk when present, trained on the event database once 50 events exist, and otherwise the heuristic is used alone, so the system works from the first transfer.

4) Weighted Risk Score

With sensitivity level L, the six signals are combined with fixed weights (Fig. 2):

$$R = 100(0.35L/3 + 0.1U + 0.2T + 0.15D + 0.1C + 0.1A) \quad (4)$$

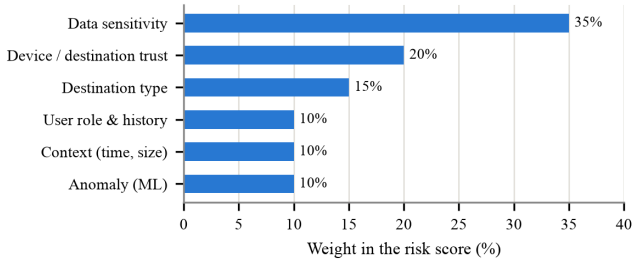


Fig. 2. Weight of each signal in the risk score. Data sensitivity dominates, followed by where the data goes (trust and destination type).

The score maps to LOW (R < 25), MEDIUM (25–50), HIGH (50–70) or CRITICAL (≥ 70). The weighted sum is fully explainable: each point can be traced to a signal, and the dashboard shows this breakdown for every event.

5) Policy Engine

The risk level is looked up in a policy table stored in the database (Table III). Three hard rules are then applied in order: Restricted data to any non-trusted destination is always blocked; Public data that would be warned about is allowed; and an administrator using a trusted device receives WARN instead of BLOCK.

TABLE III
POLICY TABLE

Risk level	Action	Effect
LOW (< 25)	ALLOW	Transfer continues and is logged
MEDIUM (25–50)	WARN	Transfer continues; amber banner; logged
HIGH (50–70)	BLOCK	Stopped on the channel; alert raised
CRITICAL (≥ 70)	BLOCK	Stopped on the channel; alert raised

6) Hash-Chained Audit Log

Each audit record r_i is stored with the hash of the previous record:

$$H_i = SHA\text{-}256(H_{i-1} \,||\, JSON(r_i)),\ H_0 = \text{“GENESIS”} \qquad (5)$$

Database triggers refuse UPDATE and DELETE on the audit table. If an attacker bypasses the triggers, verification recomputes the chain and also checks the identifier counter and that every transfer event still has its unchanged record. This catches edited rows, deleted rows (including the last one) and altered decisions.

D. Pseudocode

Algorithms 1–4 give the main procedures as implemented.

Algorithm 1: Process a transfer event

```
PROCEDURE ProcessTransfer(user, dest, payload, channel, t)
  meta   <- {name, size, SHA256(payload), t, channel}
  L, why <- Classify(payload.text, payload.name)  // L: 0..3
  U <- min(1, Role(user) + 0.1*min(Blocks30d(user), 3))
  T <- Trust(dest);  D <- DestRisk(dest.type)
  C <- 0.6 if weekend or hour not in [7,20) IST else 0
  C <- min(1, C + 0.3 if size > 50 MB)
  n <- TransfersLastHour(user, t)
  A <- AnomalyScore([hour, L, sizeMB, D, n])          // Alg. 3
  R <- 100*(0.35*L/3 + 0.10*U + 0.20*T + 0.15*D
          + 0.10*C + 0.10*A)
  band   <- LOW if R<25, MEDIUM if R<50,
            HIGH if R<70, else CRITICAL
  action <- PolicyTable[band]
  IF L = 3 AND dest.trust != trusted THEN action <- BLOCK
  ELSE IF L = 0 AND action = WARN THEN action <- ALLOW
  ELSE IF user.role = admin AND dest.trust = trusted
         AND action = BLOCK THEN action <- WARN
  SaveEvent(meta, L, R, band, action, why)
  IF action = BLOCK THEN RaiseAlert(user, dest, band, R)
  AppendAudit({meta, L, R, band, action, why})  // Alg. 4
  RETURN action, R, why
```

Algorithm 2: Classify content (rules + Luhn)

```
FUNCTION Classify(text, name)
  level <- 0; why <- []
  FOR (pattern, lvl, label) IN FilenameRules DO
    IF pattern matches name THEN
      level <- max(level, lvl); why.add(label)
  FOR (label, regex, lvl, minCount, check) IN ContentRules DO
    hits <- all matches of regex in text
    IF check exists THEN hits <- [h IN hits WHERE check(h)]
    IF |hits| >= minCount THEN
      level <- max(level, lvl)
      why.add(label + " x" + |hits|)   // type and count only
  RETURN level, why

FUNCTION Luhn(number)
  d <- digits of number, right to left; s <- 0
  FOR i, x IN d DO
    IF i is odd THEN x <- 2x; IF x > 9 THEN x <- x - 9
    s <- s + x
  RETURN 13 <= |d| <= 19 AND s mod 10 = 0
```

Algorithm 3: Train and score the anomaly detector

```
PROCEDURE Train(X)                        // no labels used
  model <- IsolationForest(trees=200, contamination=0.05).fit(X)

FUNCTION AnomalyScore(x)
  h <- min(1, x.count1h / 10)          // heuristic
  IF model is available THEN
    u <- clip((-model.score_samples(x) - 0.5) / 0.2, 0, 1)
    RETURN max(h, u)                        // hybrid
  RETURN h
```

Algorithm 4: Hash-chained audit log

```
PROCEDURE AppendAudit(record)      // one DB transaction
  prev <- hash of last audit row, or "GENESIS"
  body <- JSON(record, sorted keys)
  INSERT (body, prev, SHA256(prev || body)) INTO audit_log

FUNCTION VerifyAudit()
  prev <- "GENESIS"
  FOR row IN audit_log ORDER BY id DO
    IF row.prev != prev OR row.hash != SHA256(prev || row.body)
      THEN RETURN TAMPERED at row.id
    prev <- row.hash
  IF id counter > last id THEN RETURN TAMPERED (rows deleted)
  IF an event has no matching record THEN RETURN TAMPERED
  RETURN INTACT
```

E. Implementation

The system is a Python 3 package (data_sentinel) run as *python -m data_sentinel <command>*, with commands for initialisation, simulation, reporting, audit verification, the multi-channel agent, single-file checks, folder watching, live monitoring, user and device registration, and the Drive scan. It uses scikit-learn, NumPy and joblib for machine learning, psutil for volume discovery, aiosmtpd for the mail gateway, the standard-library HTTP server for the scan API, Streamlit with pandas and Altair for the dashboard, and matplotlib for the evaluation figures. The browser extension is written in plain JavaScript. The experiments ran on an Apple-silicon Mac with Python 3.13 and scikit-learn 1.9.

F. Output and Results

1) End-to-End Scenarios

Ten scenarios cover every channel. The *simulate* command runs them on Monday, 5 October 2026: A–C and E–J at 11:30 IST, then D as twelve copies of one file, one per minute from 22:30. Users are alice (employee), bob (admin) and carol (contractor); all files are synthetic. Table IV and Fig. 3 give the results.

TABLE IV
RESULTS OF THE TEST SCENARIOS

ID	Scenario	Channel	Sensitivity	Score	Level	Decision
A	Public notice → company laptop	Internal	Public	4.0	LOW	ALLOW
B	Internal memo → unknown USB	USB	Internal	41.2	MEDIUM	WARN
C	Payroll with test card → untrusted USB	USB	Restricted	74.0	CRITICAL	BLOCK
D	12th ordinary file copied at 22:41	USB	Internal	59.2	HIGH	BLOCK
E	Customer list → Google Drive	Web (cloud)	Confidential	50.3	HIGH	BLOCK
F	API key → ChatGPT	Web (AI chat)	Restricted	67.2	HIGH	BLOCK
G	Salary sheet → personal Gmail	E-mail	Confidential	55.3	HIGH	BLOCK
H	Lunch e-mail → colleague	Internal e-mail	Public	11.0	LOW	ALLOW
I	Restricted file → OneDrive folder	Cloud sync	Restricted	65.2	HIGH	BLOCK
J	Payroll shared publicly (Drive)	Drive sharing	Restricted	73.7	CRITICAL	BLOCK

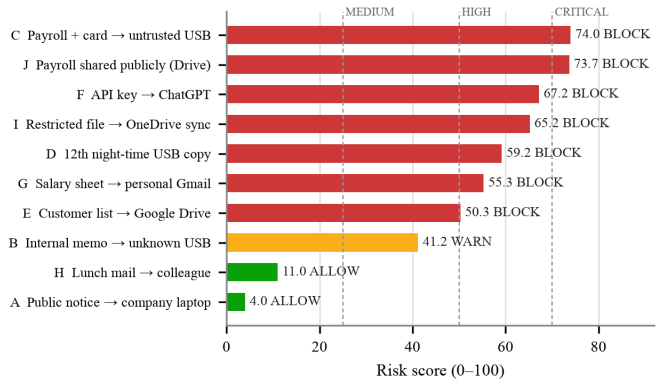


Fig. 3. Risk score and decision for each test scenario. Dashed lines mark the MEDIUM, HIGH and CRITICAL bands.

Explanation: all ten scenarios produced the expected decision. Harmless transfers (A, H) were allowed and the internal memo to an unknown USB (B) was warned. Every transfer of sensitive data was blocked on every channel. Scenario C shows the score breakdown: Restricted content gives 35 points, the untrusted device 20, the USB destination 10.5, the contractor role 7, office hours 0 and the Isolation Forest 1.5, for a total of 74.0 (CRITICAL). Scenario D shows the value of behaviour analysis: the file is only Internal (11.7 points). The unknown USB (14) and the USB destination (10.5) add 24.5, and the late hour (+6), the eleven earlier copies in the same hour (anomaly 1.0, +10) and the user's earlier blocks (+3) lift the twelfth copy to 59.2, into the HIGH band. Fig. 4 shows the command-line output for scenario C; only the type and count of the matched data appear.

```
=== C: restricted file -> untrusted USB ===
  Sensitivity : Restricted
  Risk        : 74.0 (CRITICAL)
  DECISION    : BLOCK
    - sensitive filename keyword -> Confidential
    - sensitivity keyword in filename -> Confidential
    - content: credit card number x1 -> Restricted
    - content: restricted marking x1 -> Restricted
    - content: financial/HR terms x1 -> Confidential
    - role 'contractor'
    - device 'Unknown-USB-77' is untrusted
    - destination: usb
    - anomaly score 0.15 (Isolation Forest, saved model)
    - policy: CRITICAL -> BLOCK
  ALERT -> security team: [CRITICAL] blocked usb transfer
           by 'carol': customer_payroll_confidential.csv
(score 74.0)
```

Fig. 4. Command-line output of the system for scenario C

2) Anomaly Detection

The detector was evaluated on 5,000 synthetic transfer events, 5% of them exfiltration. Normal events follow office-hour behaviour: the hour is drawn from N(13, 2.5), sensitivity levels 0–3 have probabilities 0.45/0.40/0.12/0.03, the size is lognormal, and the hourly count follows Poisson(1.5). Exfiltration events come in three equal patterns: a night-time bulk copy (8–39 transfers per hour), a single large sensitive file of 60–900 MB during office hours, and a burst of 12–59 small files. The data was split 70/30, stratified by pattern, with seed 42. The test set holds 1,500 events and 75 leaks. The threshold for all three approaches is 0.5 (Table V, Fig. 5).

TABLE V
DETECTION PERFORMANCE ON THE SYNTHETIC TEST SET

Approach	Prec.	Recall	F1	AUC	Caught
Rule-based heuristic	66.7	66.7	66.7	79.45	50/75
Isolation Forest	98.6	90.7	94.4	99.96	68/75
Proposed hybrid	73.7	97.3	83.9	99.68	73/75

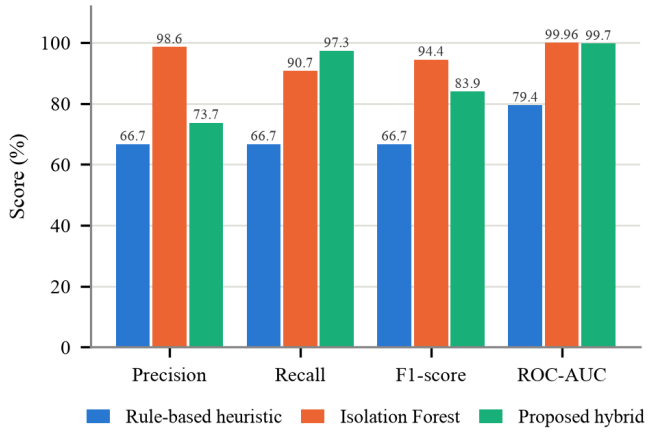


Fig. 5. Precision, recall, F1-score and ROC-AUC (%) of the rule-based heuristic, the Isolation Forest and the proposed hybrid.

Explanation: the heuristic flags only users with many transfers in an hour. It caught all 25 night-time bulk copies and all 25 bursts, but none of the 25 single large files, so its recall is 66.7%. The Isolation Forest learns normal behaviour across all five features together. It caught 23 of the 25 large files with only one false alarm, raising the F1-score from 66.7% to 94.4% and the ROC-AUC from 79.4% to 99.96%. Its weakness was the burst pattern (20 of 25). The hybrid takes the stronger signal, so it inherits the heuristic's bursts and the forest's large files, and reaches the highest recall: 73 of 75 leaks, or 97.3%. It pays for this with lower precision (26 false alarms against 1). In DLP a missed leak costs far more than a

false alarm, and because the anomaly signal is only 10% of the risk score, an anomaly alone rarely blocks a harmless transfer.

3) Comparison of Anomaly-Detection Algorithms

Five unsupervised detectors were trained on the same data and averaged over seeds 1–5 (Fig. 6). The one-class models use their own decision rule (contamination or ν = 0.05), and the ROC-AUC uses each model's continuous score.

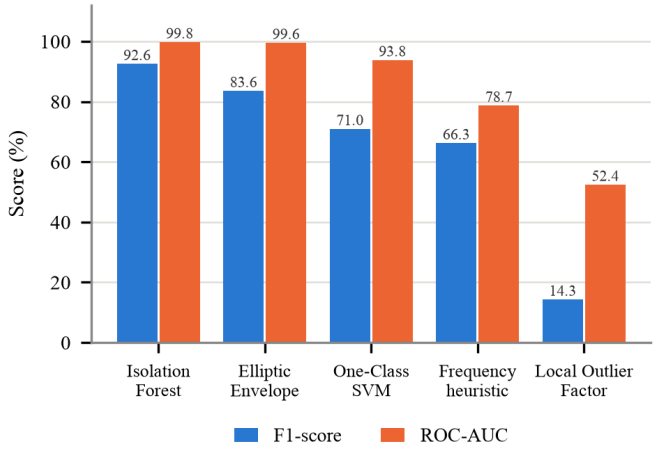


Fig. 6. F1-score and ROC-AUC (%) of five anomaly detectors, mean of five runs.

Explanation: the Isolation Forest gave the best F1-score (92.6%) and ROC-AUC (99.8%) and trained in about 108 ms. Elliptic Envelope came close in ROC-AUC (99.6%) but had a lower F1-score (83.6%) and assumes Gaussian data. One-Class SVM reached 71.0% and 93.8%, the frequency heuristic 66.3% and 78.7%, and Local Outlier Factor only 14.3% and 52.4%. Exfiltration events form small, dense groups, which look normal to a neighbourhood-based method. This ranking agrees with Peccatiello *et al.* [10], where the Isolation Forest outperformed Elliptic Envelope and Local Outlier Factor. It also supports the survey finding that unsupervised methods suit settings without labelled attacks [8].

4) Luhn Checksum

A card-number pattern alone matches any long run of digits, such as order numbers. On 10,000 random 16-digit numbers (Fig. 7), the regular expression alone raised 10,000 false alarms; with the Luhn check only 980 remained, a reduction of 90.2%. All six standard test card numbers (Visa, Mastercard, American Express, Discover, JCB and Diners Club) were still detected.

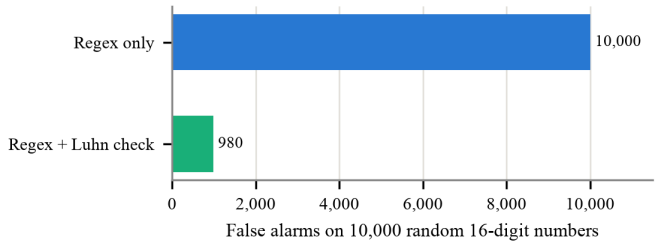


Fig. 7. False card-number alarms on 10,000 random 16-digit numbers with and without the Luhn check.

5) Channel Tests, Audit Integrity and Dashboard

With the agent running (browser API, mail gateway and a demonstration folder watched as an untrusted USB drive), each channel was exercised with synthetic files (Table VI). The tamper test changed one stored BLOCK record to ALLOW in a copy of the database; verification reported tampering at that exact row, while the original chain remained intact. Automated tests also cover editing a record, deleting a middle or final record, and changing a decision in the events table.

TABLE VI
LIVE CHANNEL TESTS

Channel	Action	Observed output
USB (demo folder)	Payroll file copied to an untrusted USB	BLOCK, 71.5 CRITICAL; file moved to quarantine/
USB (demo folder)	Internal memo copied	WARN, 47.2 MEDIUM; file kept
Browser API	API key pasted into chatgpt.com	BLOCK, 69.0 HIGH; paste dropped, red banner
SMTP gateway	Payroll attached, sent to gmail.com	550 5.7.1 Blocked by Data Sentinel
SMTP gateway	Lunch mail to a company address	250; X-Data-Sentinel header; saved/relayed
Drive scanner	Payroll sheet shared with anyone	BLOCK, 75.3; only the public link removed
Dry-run mode	Payroll file copied	BLOCK logged and alerted; file left in place
Extension (Chromium)	Upload, hidden input, drop, paste	9 end-to-end tests passed

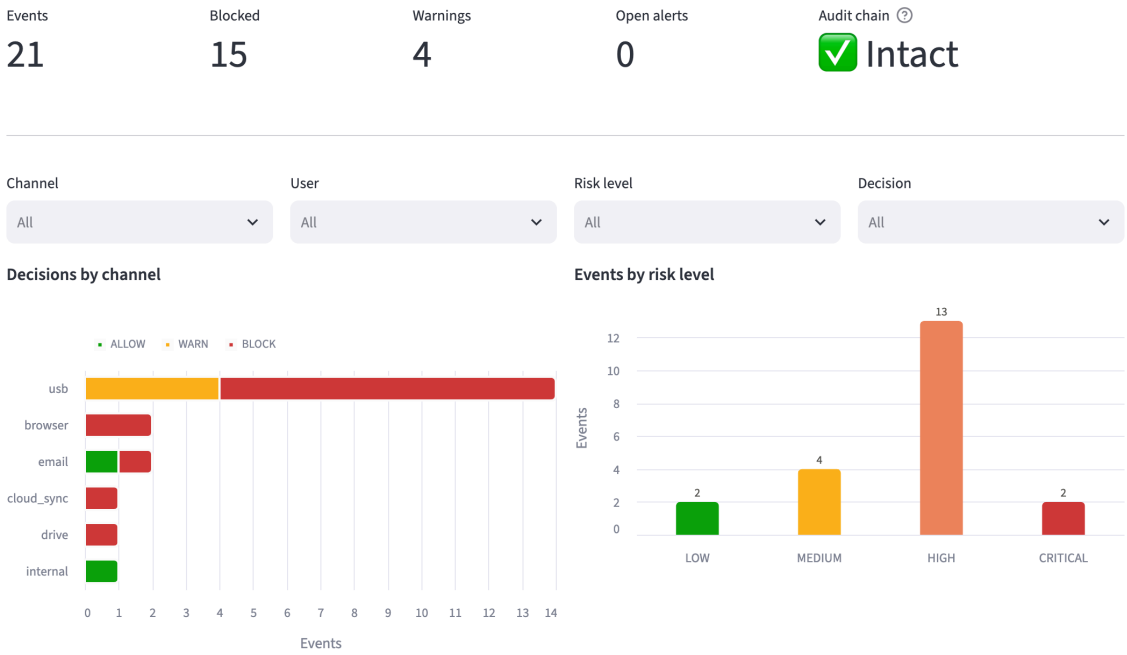


Fig. 8. Monitoring dashboard after the scenario run: totals, audit-chain status, decisions by channel and events by risk level.

Explanation: Fig. 8 shows the dashboard after the scenario run: 21 events, 15 blocks, 4 warnings and an intact audit chain. USB carries most events because of the twelve copies in scenario D. For any event, the dashboard lists the reasons and the points contributed by each signal, offers filters, lets an analyst resolve alerts, and can check an uploaded file. In total, 125 automated tests pass. They cover the classifier, the engine and policy rules, audit tampering, thread safety, the watchers, the scan API, the SMTP gateway, the Drive scanner with a fake client, the dashboard, and the browser extension end-to-end in Chromium.

V. Conclusion and Future Work

Data Sentinel AI shows that one explainable pipeline can protect sensitive data across the many routes by which it leaves an endpoint: USB drives, cloud sync folders, network shares, website uploads and pastes (including AI chat tools), desktop e-mail and over-shared Google Drive files. Combining rule-based classification with an unsupervised Isolation Forest outperformed rules alone. On synthetic data, the hybrid raised recall from 66.7% to 97.3%, and the Isolation Forest was the strongest of the five detectors compared. The Luhn check removed about 90% of false card alarms, every decision came with its reasons, and the hash-chained audit log made every decision tamper-evident.

The prototype has known limits. Copies to USB drives, sync folders and shares are handled just after the file is written, not before. Browser coverage depends on the extension, which supports Chromium-based browsers. Desktop e-mail is covered only when sent through the gateway. The machine-learning results come from synthetic data. Future work includes:

- training and validating the detector on real, labelled transfer logs and on public insider-threat data such as the CERT dataset used in [9];
- blocking file copies before they complete with operating-system file-system filter drivers;
- learning-based sensitive-data classification and OCR for images and scanned documents, building on resources such as SPEDAC [12];
- per-feature explanations for the anomaly component in the style of SHAP and LIME [7], and evaluating the deep isolation forest [11];
- anchoring the audit chain in a verifiable external log [18], integrating with zero-trust access control [14], covering OneDrive and SharePoint through the Microsoft Graph API, and tuning the risk weights from real incidents.

Acknowledgment

The authors thank their guide, Dr. Gayathri A., for the guidance and support given throughout this project, carried out for the course Python Programming (PAAML503).

References

[1] F. R. Alzaabi and A. Mehmood, “A Review of Recent Advances, Challenges, and Opportunities in Malicious Insider Threat Detection Using Machine Learning Methods,” *IEEE Access*, vol. 12, pp. 30907–30927, 2024, doi: 10.1109/ACCESS.2024.3369906.

[2] F. Whitelaw, J. Riley, and N. Elmrabit, “A Review of the Insider Threat, a Practitioner Perspective Within the U.K. Financial Services,” *IEEE Access*, vol. 12, pp. 34752–34768, 2024, doi: 10.1109/ACCESS.2024.3373265.

[3] A. Golda *et al.*, “Privacy and Security Concerns in Generative AI: A Comprehensive Survey,” *IEEE Access*, vol. 12, pp. 48126–48144, 2024, doi: 10.1109/ACCESS.2024.3381611.

[4] I. Gupta, A. K. Singh, C.-N. Lee, and R. Buyya, “Secure Data Storage and Sharing Techniques for Data Protection in Cloud Environments: A Systematic Review, Analysis, and Future Directions,” *IEEE Access*, vol. 10, pp. 71247–71277, 2022, doi: 10.1109/ACCESS.2022.3188110.

[5] N. Capuano, G. Fenza, V. Loia, and C. Stanzione, “Explainable Artificial Intelligence in CyberSecurity: A Survey,” *IEEE Access*, vol. 10, pp. 93575–93600, 2022, doi: 10.1109/ACCESS.2022.3204171.

[6] G. Rjoub *et al.*, “A Survey on Explainable Artificial Intelligence for Cybersecurity,” *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 5115–5140, Dec. 2023, doi: 10.1109/TNSM.2023.3282740.

[7] K. Saeed Alketbi and A. Mehmood, “A Comprehensive Survey of Explainable Artificial Intelligence Techniques for Malicious Insider Threat Detection,” *IEEE Access*, vol. 13, pp. 121772–121798, 2025, doi: 10.1109/ACCESS.2025.3587114.

[8] A. Umar and I. Jordanov, “Anomaly Detection in IoT Environment Using Machine Learning: A Survey,” *IEEE Internet Things J.*, vol. 13, no. 10, pp. 20477–20497, May 2026, doi: 10.1109/JIOT.2026.3671102.

[9] T. Al-Shehari, M. Al-Razgan, T. Alfakih, R. A. Alsowail, and S. Pandiaraj, “Insider Threat Detection Model Using Anomaly-Based Isolation Forest Algorithm,” *IEEE Access*, vol. 11, pp. 118170–118185, 2023, doi: 10.1109/ACCESS.2023.3326750.

[10] R. B. Peccatiello, J. J. C. Gondim, and L. P. F. Garcia, “Applying One-Class Algorithms for Data Stream-Based Insider Threat Detection,” *IEEE Access*, vol. 11, pp. 70560–70573, 2023, doi: 10.1109/ACCESS.2023.3293825.

[11] H. Xu, G. Pang, Y. Wang, and Y. Wang, “Deep Isolation Forest for Anomaly Detection,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12591–12604, Dec. 2023, doi: 10.1109/TKDE.2023.3270293.

[12] G. Gambarelli, A. Gangemi, and R. Tripodi, “Is Your Model Sensitive? SPEDAC: A New Resource for the Automatic Classification of Sensitive Personal Data,” *IEEE Access*, vol. 11, pp. 10864–10880, 2023, doi: 10.1109/ACCESS.2023.3240089.

[13] D. Oreščanin, T. Hlupić, and B. Vrdoljak, “Managing Personal Identifiable Information in Data Lakes,” *IEEE Access*, vol. 12, pp. 32164–32180, 2024, doi: 10.1109/ACCESS.2024.3365042.

[14] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, “Zero Trust Architecture (ZTA): A Comprehensive Survey,” *IEEE Access*, vol. 10, pp. 57143–57179, 2022, doi: 10.1109/ACCESS.2022.3174679.

[15] H. Oz, D. C. D’Elia, G. S. Tuncay, A. Acar, R. Lazzeretti, and S. Uluagac, “With Great Power Comes Great Responsibility: Security and Privacy Issues of Modern Browser Application Programming Interfaces,” *IEEE Security Privacy*, vol. 23, no. 2, pp. 52–60, Mar. 2025, doi: 10.1109/MSEC.2024.3443625.

[16] X. Cai, H. Zhang, C. M. Ahmed, and H. Koide, “Detecting Advanced Persistent Threat Exfiltration With Ensemble Deep Learning Tree Models and Novel Detection Metrics,” *IEEE Access*, vol. 13, pp. 81803–81822, 2025, doi: 10.1109/ACCESS.2025.3567772.

[17] J. Henriques, F. Caldeira, T. Cruz, and P. Simões, “A Survey on Forensics and Compliance Auditing for Critical Infrastructure Protection,” *IEEE Access*, vol. 12, pp. 2409–2444, 2024, doi: 10.1109/ACCESS.2023.3348552.

[18] X. Huang *et al.*, “BLAS: A Blockchain-Enabled Efficient and Verifiable Log Audit System With Hybrid Storage,” *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 4, pp. 8231–8248, Jul. 2026, doi: 10.1109/TDSC.2026.3681195.